

Section 13.2 — DOM Events & Event Propagation

JavaScript দিয়ে webpage-কে interactive করার সবচেয়ে গুরুত্বপূর্ণ foundation হলো **DOM Events** ।

Button click করা, input-এ লেখা, form submit করা, keyboard press করা, mouse move করা—সবকিছুই browser-এর কাছে কোনো না কোনো **event** ।

এই document-এ আমরা শিখব:

1. Event কী
 2. `addEventListener()`
 3. Event listener কীভাবে কাজ করে
 4. Callback function এবং event
 5. Common DOM events
 6. `removeEventListener()`
 7. Function reference কেন গুরুত্বপূর্ণ
 8. Event object
 9. `event.target`
 10. `event.currentTarget`
 11. `event.type`
 12. `preventDefault()`
 13. `stopPropagation()`
 14. Event propagation
 15. Event bubbling
 16. Event capturing
 17. Target phase
 18. Event propagation order
 19. `capture: true`
 20. Bubbling vs capturing
 21. Event delegation
 22. Practical event delegation
 23. `target` vs `currentTarget`
 24. Common mistakes
 25. Best practices
 26. Practice problems
 27. Interview questions
 28. Quick revision
 29. Mental model
-

1. Event কী?

Browser-এ কোনো ঘটনা ঘটলে সেটাকে **event** বলা হয়।

যেমন:

```
Button click
Input change
Form submit
Keyboard press
Mouse move
Page load
Window resize
```

Example:

```
<button id="btn">Click Me</button>
```

User button-এ click করলে browser একটি `click` event তৈরি করে।

JavaScript সেই event শুনতে পারে।

```
const button = document.querySelector("#btn");

button.addEventListener("click", () => {
  console.log("Button clicked");
});
```

Mental model:

```
User action
  ↓
Browser detects event
  ↓
Event object তৈরি
  ↓
Event listener execute
  ↓
JavaScript response
```

2. addEventListener()

কোনো element-এর event শুনতে সবচেয়ে common method:

```
element.addEventListener(eventType, callback);
```

Example:

```
const button = document.querySelector("#btn");

button.addEventListener("click", () => {
  console.log("Clicked!");
});
```

এখানে:

```
addEventListener
├── "click"
└── callback function
```

3. addEventListener() Syntax

```
element.addEventListener("event", function);
```

Example:

```
button.addEventListener("click", handleClick);
```

এখানে:

```
"click"
→ event type
```

```
handleClick  
→ callback/listener function
```

4. Function Pass করা vs Function Call করা

এটি খুব important ।

সঠিক:

```
button.addEventListener("click", handleClick);
```

ভুল:

```
button.addEventListener("click", handleClick());
```

কারণ:

```
handleClick
```

মানে function reference ।

কিন্তু:

```
handleClick()
```

মানে function এখনই execute করে ।

`addEventListener()` -কে callback দিতে হয়, immediate result নয় ।

Mental model:

```
handleClick  
→ "এই function পরে চালাবে"  
  
handleClick()  
→ "এই মুহূর্তে function চালাও"
```

5. Named Callback

```
function handleClick() {  
  console.log("Button clicked");  
}  
  
button.addEventListener("click", handleClick);
```

এটি ভালো approach যখন একই function অন্য জায়গাতেও ব্যবহার করা হবে বা পরে remove করতে হবে।

6. Anonymous Callback

চাইলে সরাসরি function দিতে পারো:

```
button.addEventListener("click", function () {  
  console.log("Clicked");  
});
```

Arrow function:

```
button.addEventListener("click", () => {  
  console.log("Clicked");  
});
```

দুটোই valid।

7. Common DOM Events

JavaScript-এ অনেক ধরনের event আছে।

Mouse events

```
click  
dblclick  
mousedown  
mouseup  
mousemove
```

```
mouseenter  
mouseleave
```

Keyboard events

```
keydown  
keyup
```

Form events

```
submit  
input  
change  
focus  
blur
```

Window events

```
load  
resize  
scroll
```

8. click

সবচেয়ে common event:

```
button.addEventListener("click", () => {  
  console.log("Clicked");  
});
```

9. input

Input-এর value change হওয়ার সাথে সাথে detect করতে:

```
const input = document.querySelector("#name");

input.addEventListener("input", () => {
  console.log(input.value);
});
```

User প্রতিবার character লিখলে callback চলবে।

10. change

`change` event সাধারণত value change হওয়ার পরে trigger হয়।

```
input.addEventListener("change", () => {
  console.log("Value changed");
});
```

`input` এবং `change` এক জিনিস নয়।

```
input
→ typing-এর সাথে সাথে

change
→ value change/commit হওয়ার পরে
```

11. submit

Form submit detect করতে:

```
const form = document.querySelector("#form");

form.addEventListener("submit", (event) => {
  console.log("Form submitted");
});
```

এখানে event object পাওয়া যাচ্ছে।

12. Event Object

Event ঘটলে browser callback function-এ একটি **event object** পাঠায়।

```
button.addEventListener("click", (event) => {  
  console.log(event);  
});
```

এই object-এর মধ্যে event সম্পর্কে অনেক information থাকে।

যেমন:

```
event.type  
event.target  
event.currentTarget  
event.preventDefault()  
event.stopPropagation()
```

13. event.type

কোন event ঘটেছে সেটা জানতে:

```
button.addEventListener("click", (event) => {  
  console.log(event.type);  
});
```

Output:

```
click
```

একইভাবে:

```
input.addEventListener("input", (event) => {  
  console.log(event.type);  
});
```

Output:

input

14. event.target

`event.target` হলো যে **actual element** থেকে **event** শুরু হয়েছে।

HTML:

```
<button id="btn">
  Click Me
</button>
```

JavaScript:

```
button.addEventListener("click", (event) => {
  console.log(event.target);
});
```

Output হবে clicked button element।

15. target এর Important Example

HTML:

```
<button id="btn">
  <span>Click Me</span>
</button>
```

যদি user `<span>`-এর উপর click করে:

```
button.addEventListener("click", (event) => {
  console.log(event.target);
});
```

`event.target` হতে পারে:

```
<span>Click Me</span>
```

অর্থাৎ `target` হলো **actual clicked element**।

16. `event.currentTarget`

`currentTarget` হলো যে **element**-এর **listener** বর্তমানে **execute** হচ্ছে।

Example:

```
button.addEventListener("click", (event) => {  
  console.log(event.target);  
  console.log(event.currentTarget);  
});
```

যদি `button`-এর ভিতরের `<span>` click করা হয়:

```
target  
→ span  
  
currentTarget  
→ button
```

এটা খুব important distinction।

17. `target` vs `currentTarget`

Property	Meaning
<code>event.target</code>	Actual event origin
<code>event.currentTarget</code>	Current listener-এর element

Mental model:

```
User clicks  
↓  
target  
↓
```

Event travels

↓

currentTarget = যেই element-এর listener এখন চলছে

18. Practical target Example

ধরো:

```
<ul id="list">
  <li>Apple</li>
  <li>Banana</li>
  <li>Orange</li>
</ul>
```

```
const list = document.querySelector("#list");

list.addEventListener("click", (event) => {
  console.log(event.target);
});
```

যদি Banana click করো:

```
event.target
→ <li>Banana</li>
```

এটি পরে **event delegation**-এ খুব important হবে।

19. preventDefault()

Browser-এর কোনো default behavior বন্ধ করতে:

```
event.preventDefault();
```

ব্যবহার করা হয়।

20. Form Example

HTML:

```
<form id="form">
  <input type="text">
  <button type="submit">Submit</button>
</form>
```

Normally form submit করলে browser page reload/navigation করতে পারে।

JavaScript:

```
const form = document.querySelector("#form");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  console.log("Custom form handling");
});
```

এখন browser-এর default form submission বন্ধ হবে।

21. Link Example

HTML:

```
<a href="https://example.com" id="link">
  Visit
</a>
```

JavaScript:

```
const link = document.querySelector("#link");

link.addEventListener("click", (event) => {
  event.preventDefault();
});
```

```
console.log("Navigation stopped");
});
```

এখানে default navigation বন্ধ করা হয়েছে।

22. preventDefault() কী করে না?

একটি important distinction:

```
event.preventDefault();
```

event propagation বন্ধ করে না।

এটি শুধু browser-এর **default action** বন্ধ করে।

অর্থাৎ:

```
preventDefault()
→ default browser behavior বন্ধ

stopPropagation()
→ event propagation বন্ধ
```

দুটো আলাদা।

23. stopPropagation()

Event-এর propagation বন্ধ করতে:

```
event.stopPropagation();
```

Example:

```
child.addEventListener("click", (event) => {
  event.stopPropagation();
});
```

এতে event parent-এর দিকে propagate করবে না।

24. Event Propagation কী?

Event একটি element-এ ঘটান পর DOM tree-এর মধ্য দিয়ে travel করতে পারে।

এটাই **event propagation**।

ধরো:

```
<div id="parent">
  <button id="child">
    Click
  </button>
</div>
```

User button click করল।

Event শুধু button-এ আটকে থাকে না।

এটি DOM hierarchy-এর মধ্য দিয়ে travel করে।

25. Event Propagation-এর 3 Phase

DOM event propagation-কে সাধারণভাবে 3 phase-এ বোঝানো হয়:

1. Capturing Phase
2. Target Phase
3. Bubbling Phase

Diagram:

```
Document
  ↓
HTML
  ↓
Body
  ↓
Parent
```

↓
Button ← Target
↑
Parent
↑
Body
↑
HTML
↑
Document

প্রথমে নিচের দিকে আসে:

Capturing

Target-এ পৌঁছে:

Target phase

তারপর উপরের দিকে ফিরে যায়:

Bubbling

26. Bubbling কী?

Default event flow সাধারণত **bubbling**-এর মাধ্যমে child থেকে parent-এর দিকে যায়।

HTML:

```
<div id="parent">  
  <button id="child">  
    Click  
  </button>  
</div>
```

JavaScript:

```
const parent = document.querySelector("#parent");  
const child = document.querySelector("#child");
```

```
parent.addEventListener("click", () => {  
  console.log("Parent");  
});  
  
child.addEventListener("click", () => {  
  console.log("Child");  
});
```

Button click করলে output:

```
Child  
Parent
```

কারণ event:

```
Child  
  ↓  
Parent
```

দিকে bubble করে।

27. Bubbling Mental Model

```
Click Button  
  ↓  
Button listener  
  ↓  
Parent listener  
  ↓  
Grandparent listener  
  ↓  
...
```

অর্থাৎ:

```
bottom → top
```

28. stopPropagation() with Bubbling

```
child.addEventListener("click", (event) => {  
  event.stopPropagation();  
  
  console.log("Child");  
});
```

Parent:

```
parent.addEventListener("click", () => {  
  console.log("Parent");  
});
```

এখন child click করলে:

Child

কিন্তু:

Parent

চলবে না।

কারণ propagation stop করা হয়েছে।

29. Capturing কী?

Capturing হলো event-এর প্রথম phase, যেখানে event **ancestor** থেকে **target**-এর দিকে আসে।

Flow:

```
Parent  
↓  
Child
```

Capturing listener enable করতে:

```
parent.addEventListener(  
  "click",  
  () => {  
    console.log("Parent");  
  },  
  { capture: true }  
);
```

30. Bubbling vs Capturing

Bubbling

```
Child  
↓  
Parent
```

Capturing

```
Parent  
↓  
Child
```

Mental model:

```
CAPTURE  
top → bottom  
  
TARGET  
actual element  
  
BUBBLE  
bottom → top
```

31. Complete Propagation Example

HTML:

```
<div id="grandparent">
  <div id="parent">
    <button id="child">
      Click
    </button>
  </div>
</div>
```

JavaScript:

```
grandparent.addEventListener(
  "click",
  () => {
    console.log("Grandparent capture");
  },
  { capture: true }
);

parent.addEventListener(
  "click",
  () => {
    console.log("Parent capture");
  },
  { capture: true }
);

child.addEventListener("click", () => {
  console.log("Child");
});

parent.addEventListener("click", () => {
  console.log("Parent bubble");
});

grandparent.addEventListener("click", () => {
  console.log("Grandparent bubble");
});
```

Button click করলে conceptual order:

```
Grandparent capture
Parent capture
Child
```

Parent bubble
Grandparent bubble

32. Event Propagation Order

Simplified:

Document
↓
HTML
↓
Body
↓
Parent
↓
Target
↑
Parent
↑
Body
↑
HTML
↑
Document

প্রথম অংশ:

Capturing

মাঝে:

Target

শেষ অংশ:

Bubbling

33. capture: true

`addEventListener()` -এর third argument দিয়ে capture phase-এ listener চালানো যায়।

```
element.addEventListener(  
  "click",  
  handleClick,  
  { capture: true }  
);
```

Default:

```
element.addEventListener(  
  "click",  
  handleClick  
);
```

সাধারণত bubbling phase-এ listener execute হয়।

34. removeEventListener()

Event listener remove করতে:

```
element.removeEventListener("click", handleClick);
```

কিন্তু এখানে একটি খুব important rule আছে।

একই **function reference** ব্যবহার করতে হবে।

35. Correct removeEventListener()

```
function handleClick() {  
  console.log("Clicked");  
}  
  
button.addEventListener("click", handleClick);
```

```
button.removeEventListener("click", handleClick);
```

এখানে listener remove হবে।

36. Incorrect `removeEventListener()`

এটি কাজ করবে না:

```
button.addEventListener("click", () => {  
  console.log("Clicked");  
});  
  
button.removeEventListener("click", () => {  
  console.log("Clicked");  
});
```

কারণ দুইটা function দেখতে একই হলেও তারা দুইটি আলাদা **function object**।

Mental model:

```
Function A  
≠  
Function B
```

37. কেন **Function Reference** দরকার?

JavaScript-এ function object-এর reference গুরুত্বপূর্ণ।

```
const handler = () => {  
  console.log("Clicked");  
};  
  
button.addEventListener("click", handler);  
  
button.removeEventListener("click", handler);
```

এখানে একই reference ব্যবহার করা হয়েছে।

38. `removeEventListener()`-এর **Capture Option**

Listener add করার সময়:

```
element.addEventListener(  
  "click",  
  handler,  
  { capture: true }  
);
```

Remove করার সময় matching capture setting দরকার:

```
element.removeEventListener(  
  "click",  
  handler,  
  { capture: true }  
);
```

39. Multiple Event Listeners

একই element-এ multiple listener থাকতে পারে।

```
button.addEventListener("click", firstHandler);  
  
button.addEventListener("click", secondHandler);
```

Click করলে দুটো callback-ই execute হতে পারে।

40. Event Listener-এ Data

Event object ব্যবহার করে অনেক information পাওয়া যায়।

```
button.addEventListener("click", (event) => {  
  console.log(event.type);  
  console.log(event.target);  
});
```

```
console.log(event.currentTarget);  
});
```

41. Keyboard Event

```
document.addEventListener("keydown", (event) => {  
  console.log(event.key);  
});
```

User press করলে:

a

বা relevant key value পাওয়া যাবে।

42. Keyboard Shortcut Example

```
document.addEventListener("keydown", (event) => {  
  if (event.key === "Escape") {  
    console.log("Escape pressed");  
  }  
});
```

43. Form Input Example

```
const input = document.querySelector("#name");  
  
input.addEventListener("input", (event) => {  
  console.log(event.target.value);  
});
```

এখানে:

```
event.target  
→ input element
```

```
event.target.value  
→ current input value
```

44. Event Delegation

এটি DOM Events-এর সবচেয়ে important practical patternগুলোর একটি।

ধরো:

```
<ul id="list">  
  <li>Apple</li>  
  <li>Banana</li>  
  <li>Orange</li>  
</ul>
```

প্রতিটি `li`-তে আলাদা listener দিতে পারো।

কিন্তু অনেক element থাকলে সেটা inefficient হতে পারে।

তার বদলে parent-এর উপর একটি listener দেওয়া যায়।

এটাই:

Event Delegation

45. Event Delegation Example

```
const list = document.querySelector("#list");  
  
list.addEventListener("click", (event) => {  
  console.log(event.target);  
});
```

এখন list-এর ভিতরের child click করলে event bubble হয়ে `list`-এ আসবে।

46. Event Delegation + `target`

```
list.addEventListener("click", (event) => {  
  const item = event.target;  
  
  if (item.tagName === "LI") {  
    console.log(item.textContent);  
  }  
});
```

User যেই `<li>` click করবে, সেটাই `event.target` হবে।

47. Better Event Delegation

`tagName` check করার বদলে `matches()` ব্যবহার করা যায়।

```
list.addEventListener("click", (event) => {  
  if (event.target.matches("li")) {  
    console.log(event.target.textContent);  
  }  
});
```

48. `closest()` with Event Delegation

বাস্তব project-এ nested elements থাকতে পারে।

HTML:

```
<li class="item">  
  <span>Apple</span>  
</li>
```

User যদি `<span>` click করে:

```
event.target
```

হবে `<span>`।

কিন্তু আমরা আসলে `<li>` চাই।

তখন:

```
const item = event.target.closest(".item");
```

এটি খুব useful।

49. Practical Event Delegation

HTML:

```
<ul id="products">
  <li class="product" data-id="101">
    <span>Laptop</span>
  </li>

  <li class="product" data-id="102">
    <span>Mouse</span>
  </li>
</ul>
```

JavaScript:

```
const products = document.querySelector("#products");

products.addEventListener("click", (event) => {
  const product = event.target.closest(".product");

  if (!product) return;

  console.log(product.dataset.id);
});
```

এখন user `<span>` বা `<li>` যেখানেই click করুক, `closest .product` পাওয়া যাবে।

50. Event Delegation কেন Useful?

ধরো 1000টা button আছে।

Traditional approach:

```
1000 buttons
↓
1000 listeners
```

Event delegation:

```
1000 buttons
↓
1 parent listener
```

এতে:

- listener count কমে
- dynamically added elements handle করা সহজ হয়
- large lists-এর ক্ষেত্রে useful
- code simpler হতে পারে

51. Dynamic Elements + Event Delegation

ধরো page load হওয়ার পরে নতুন `<li>` তৈরি হলো।

যদি parent-এ delegation থাকে:

```
list.addEventListener("click", (event) => {
  if (event.target.matches("li")) {
    console.log("Clicked");
  }
});
```

পরে নতুন `li` add করলেও parent listener সেটাকে handle করতে পারবে।

কারণ event নতুন child থেকে bubble করে parent-এ যাবে।

52. target vs currentTarget in Delegation

```
list.addEventListener("click", (event) => {  
  console.log(event.target);  
  console.log(event.currentTarget);  
});
```

যদি `li` click করে:

```
target  
→ clicked li  
  
currentTarget  
→ list
```

এটি event delegation বুঝতে অত্যন্ত important।

53. preventDefault() vs stopPropagation()

Method	কাজ
<code>preventDefault()</code>	Browser-এর default action বন্ধ
<code>stopPropagation()</code>	Event propagation বন্ধ

Example:

```
event.preventDefault();
```

→ link navigation/form submission prevent করতে পারে।

```
event.stopPropagation();
```

→ parent/ancestor listener-এ event যেতে বাধা দিতে পারে।

একটি অন্যটির replacement নয়।

54. `stopPropagation()` vs `stopImmediatePropagation()`

Advanced কিন্তু useful concept:

```
event.stopImmediatePropagation();
```

এটি:

1. Event propagation stop করে
2. একই element-এর অন্য event listener-গুলোর execution-ও stop করতে পারে

Example:

```
button.addEventListener("click", (event) => {  
  event.stopImmediatePropagation();  
});
```

সাধারণ application code-এ এটি খুব কম দরকার হয়।

55. Event Propagation বন্ধ করা সবসময় ভালো নয়

অনেক beginner সব জায়গায়:

```
event.stopPropagation();
```

ব্যবহার করতে চায়।

এটা ভালো practice নয়।

কারণ:

- event delegation ভেঙে যেতে পারে
- parent components/listeners event পাবে না
- debugging কঠিন হতে পারে
- অন্য code-এর expected behavior নষ্ট হতে পারে

শুধু প্রয়োজন হলে propagation stop করো।

56. Event Listener Cleanup

Modern applications-এ event listener cleanup important।

বিশেষ করে:

- SPA
- component-based application
- dynamic DOM
- React `useEffect`

Conceptually:

```
element.addEventListener("click", handler);  
  
// later  
element.removeEventListener("click", handler);
```

React-এ এই concept cleanup function-এর সাথে সম্পর্কিত।

57. React-এর সাথে Connection

React-এ সাধারণত:

```
<button onClick={handleClick}>  
  Click  
</button>
```

Vanilla DOM:

```
button.addEventListener("click", handleClick);
```

Concept একই:

```
User click  
  ↓  
Event  
  ↓  
Handler
```

↓
UI/state change

React নিজস্ব event system ব্যবহার করে, কিন্তু native DOM event concepts জানা থাকলে React events বুঝতে অনেক সহজ হয়।

58. Event Handling + Form Validation

Real-world example:

```
form.addEventListener("submit", (event) => {  
  event.preventDefault();  
  
  const email = emailInput.value.trim();  
  
  if (!email) {  
    console.log("Email is required");  
    return;  
  }  
  
  console.log("Form submitted");  
});
```

Flow:

```
Submit  
↓  
preventDefault()  
↓  
Read input  
↓  
Validate  
↓  
Success / Error
```

59. Real-World Example — Modal

HTML:

```
<div id="modal">
  <div id="modal-content">
    <button id="close">Close</button>
  </div>
</div>
```

Modal background click করলে close করতে পারে:

```
modal.addEventListener("click", (event) => {
  if (event.target === modal) {
    modal.classList.remove("open");
  }
});
```

এখানে:

```
event.target === modal
```

মানে user actual background-এ click করেছে।

Content-এর ভিতরে click করলে modal close হবে না।

60. Common Mistake — target vs currentTarget

অনেকে ভাবে:

```
event.target
```

সবসময় সেই element যেটাতে listener দেওয়া হয়েছে।

এটা ভুল।

Listener:

```
parent.addEventListener("click", handler);
```

Child click হলে:

```
target
→ child

currentTarget
→ parent
```

61. Common Mistake — `preventDefault()` সব **propagation** বন্ধ করে ভাবা

ভুল ধারণা:

```
event.preventDefault();
```

মানে event parent-এ যাবে না।

না।

সঠিক:

```
preventDefault()
→ default browser action বন্ধ

stopPropagation()
→ propagation বন্ধ
```

62. Common Mistake — `stopPropagation()` দিয়ে সব সমস্যা **solve** করা

```
event.stopPropagation();
```

অতিরিক্ত ব্যবহার করলে event delegation এবং parent-level behavior নষ্ট হতে পারে।

তাই আগে বুঝতে হবে:

আমার কি default action বন্ধ করতে হবে?

→ `preventDefault()`

আমার কি event propagation বন্ধ করতে হবে?

→ `stopPropagation()`

63. Common Mistake — Anonymous Function Remove করা

ভুল:

```
button.addEventListener("click", () => {  
  console.log("Hi");  
});  
  
button.removeEventListener("click", () => {  
  console.log("Hi");  
});
```

কারণ দুই callback আলাদা function reference।

সঠিক:

```
function handleClick() {  
  console.log("Hi");  
}  
  
button.addEventListener("click", handleClick);  
  
button.removeEventListener("click", handleClick);
```

64. Common Mistake — Function Immediately Execute করা

ভুল:

```
button.addEventListener("click", handleClick());
```

সঠিক:

```
button.addEventListener("click", handleClick);
```

65. Common Mistake — onclick এবং addEventListener() Confuse করা

এটাও করা যায়:

```
button.onclick = handleClick;
```

কিন্তু modern code-এ addEventListener() বেশি flexible:

```
button.addEventListener("click", handleClick);
```

কারণ একই event-এর জন্য multiple listeners রাখা যায়।

66. Event Listener Options

addEventListener() -এর third argument-এ options দেওয়া যায়।

```
element.addEventListener(  
  "click",  
  handler,  
  {  
    capture: true,  
    once: true,  
    passive: true  
  }  
);
```

Important options:

capture

```
{ capture: true }
```

→ capturing phase-এ listener।

once

```
{ once: true }
```

→ listener একবার execute হওয়ার পর automatically remove হবে।

passive

```
{ passive: true }
```

→ listener `preventDefault()` করবে না বলে browser-কে signal দেয়; এটি বিশেষ করে কিছু scrolling/touch-related event-এ useful।

67. once: true

Example:

```
button.addEventListener(  
  "click",  
  () => {  
    console.log("Only once");  
  },  
  { once: true }  
);
```

প্রথম click:

```
Only once
```

দ্বিতীয় click:

কিছু হবে না

68. Event Delegation-এর Golden Pattern

```
parent.addEventListener("click", (event) => {  
  const target = event.target.closest(".item");  
  
  if (!target) return;  
  
  // Handle item  
});
```

এটি dynamic list/UI-এর জন্য খুব useful pattern ।

69. Practice — Basic Events

HTML:

```
<button id="btn">Click</button>
```

Task:

Button click করলে:

Button clicked

console-এ দেখাও ।

70. Practice — Event Object

Button click হলে console-এ দেখাও:

```
event.type  
event.target  
event.currentTarget
```

71. Practice — Form

একটি form submit হলে:

1. Page reload prevent করো
2. Input value console-এ দেখাও

72. Practice — Class Toggle

Button click করলে `.active` class toggle করো।

```
button.addEventListener("click", () => {  
  element.classList.toggle("active");  
});
```

73. Practice — Bubbling

HTML:

```
<div id="parent">  
  <button id="child">  
    Click  
  </button>  
</div>
```

দুটোতেই click listener বসে।

Output observe করো:

```
Child  
Parent
```

74. Practice — Stop Propagation

উপরের example-এ এমনভাবে modify করো যেন:

```
Child
```

শুধু print হয়।

75. Practice — Capturing

Parent-এর listener:

```
{ capture: true }
```

দিয়ে observe করো event order কীভাবে পরিবর্তন হয়।

76. Practice — Event Delegation

HTML:

```
<ul id="list">
  <li>Apple</li>
  <li>Banana</li>
  <li>Orange</li>
</ul>
```

শুধু `<ul>` -এ একটি listener দিয়ে কোন `<li>` click হয়েছে সেটা বের করো।

77. Practice — Dynamic List

JavaScript দিয়ে নতুন `<li>` add করো:

```
const li = document.createElement("li");
```

```
li.textContent = "Mango";  
  
list.append(li);
```

তারপর দেখো parent-level event delegation ব্যবহার করলে নতুন item-ও automatically কাজ করছে কিনা।

78. Interview Questions

1. Event কী?

Browser-এ ঘটে যাওয়া কোনো user/browser action বা ঘটনা।

2. `addEventListener()` কেন ব্যবহার করি?

কোনো element-এর event listen করে callback execute করার জন্য।

3. `event.target` কী?

যে actual element থেকে event শুরু হয়েছে।

4. `event.currentTarget` কী?

যে element-এর listener বর্তমানে execute হচ্ছে।

5. `target` এবং `currentTarget` এর difference কী?

```
target  
→ event-এর actual origin  
  
currentTarget  
→ current listener-এর element
```

6. `preventDefault()` কী করে?

Browser-এর default action prevent করে।

7. `stopPropagation()` কী করে?

Event-এর propagation stop করে।

8. Event bubbling কী?

Event target থেকে ancestor-এর দিকে propagate করা।

Child → Parent → Grandparent

9. Event capturing কী?

Event ancestor থেকে target-এর দিকে propagate করা।

Parent → Child

10. Event propagation-এর phases কী?

Capturing
↓
Target
↓
Bubbling

11. `removeEventListener()` কেন কাজ করে না অনেক সময়?

কারণ listener remove করার সময় একই function reference ব্যবহার করা হয়নি।

12. Event delegation কী?

Parent-এর একটি listener ব্যবহার করে তার child elements-এর events handle করার technique।

13. Event delegation কেন useful?

বিশেষ করে:

- large lists
- dynamic elements
- কম listeners
- simpler event handling

এর জন্য useful।

14. `preventDefault()` কি **propagation stop** করে?

না।

```
preventDefault()  
→ default action  
  
stopPropagation()  
→ propagation
```

79. Quick Revision

Event Listener

```
element.addEventListener("click", handler);
```

Remove

```
element.removeEventListener("click", handler);
```

Event Object

```
event.type  
event.target  
event.currentTarget
```

Prevent Default

```
event.preventDefault();
```

Stop Propagation

```
event.stopPropagation();
```

Capture

```
{  
  capture: true  
}
```

Once

```
{  
  once: true  
}
```

Event Delegation

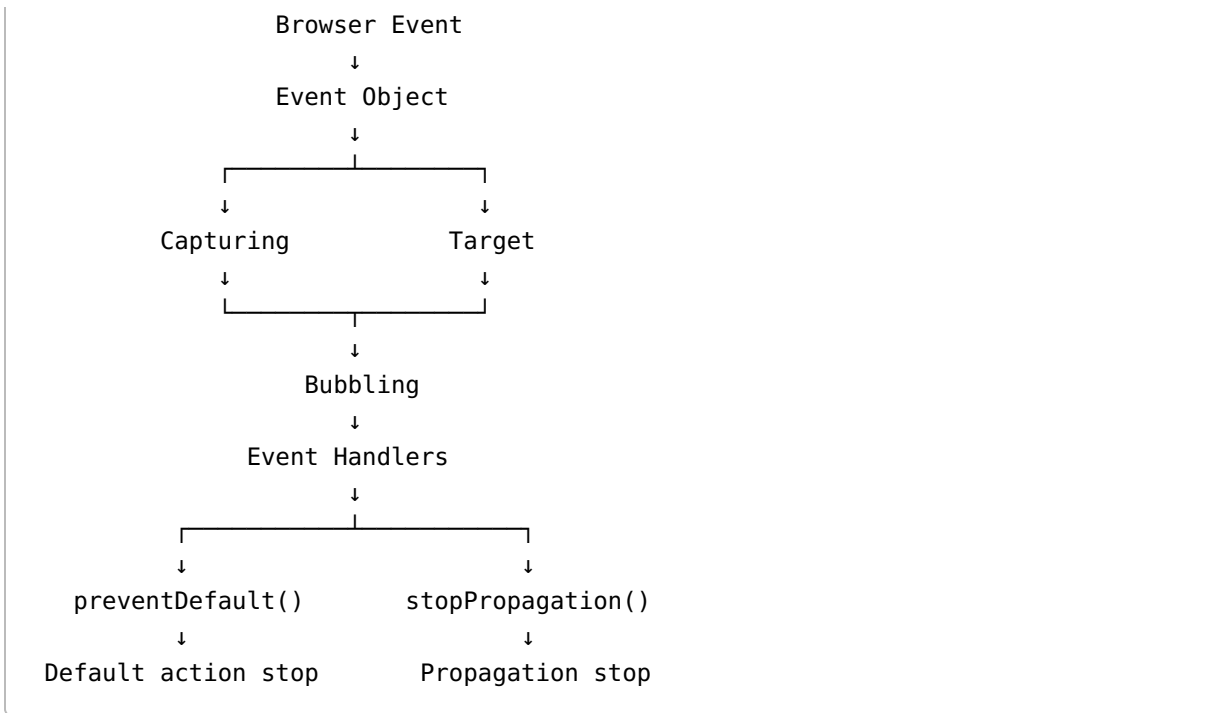
```
parent.addEventListener("click", (event) => {  
  const item = event.target.closest(".item");  
  
  if (!item) return;  
  
  // handle  
});
```

80. Complete Event Mental Model

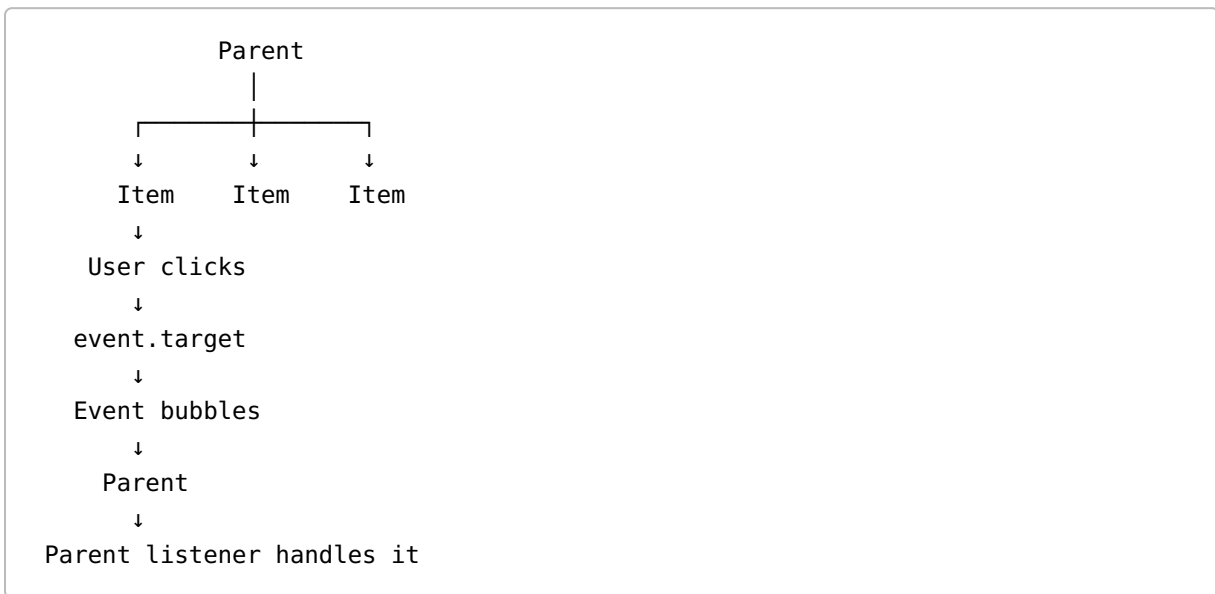
পুরো DOM Event system-কে এভাবে মনে রাখো:

```
USER ACTION
```

```
↓
```



81. Event Delegation Mental Model



82. Final Golden Rules

Rule 1: Event handle করার modern foundation হলো `addEventListener()` ।

Rule 2: `handleClick` এবং `handleClick()` এক জিনিস নয়।

Rule 3: `event.target` = actual event origin।

Rule 4: `event.currentTarget` = current listener-এর element।

Rule 5: `preventDefault()` default browser behavior বন্ধ করে।

Rule 6: `stopPropagation()` event propagation বন্ধ করে।

Rule 7: Bubbling = child → parent।

Rule 8: Capturing = parent → child।

Rule 9: Event flow = Capturing → Target → Bubbling।

Rule 10: `removeEventListener()` করতে একই function reference ব্যবহার করো।

Rule 11: Event delegation bubbling-এর উপর heavily নির্ভর করে।

Rule 12: Dynamic lists-এর জন্য event delegation খুব powerful pattern।

Rule 13: `stopPropagation()` অপ্রয়োজনে ব্যবহার করো না।



Section 13.2 Mastery Checklist

- ☐ Event কী বুঝি
- ☐ `addEventListener()` পারি
- ☐ Callback দিয়ে event handle করতে পারি
- ☐ Function reference vs function call বুঝি
- ☐ Common DOM events জানি
- ☐ `removeEventListener()` পারি
- ☐ Same function reference কেন দরকার বুঝি
- ☐ Event object বুঝি
- ☐ `event.type` বুঝি
- ☐ `event.target` বুঝি
- ☐ `event.currentTarget` বুঝি
- ☐ `target` vs `currentTarget` explain করতে পারি
- ☐ `preventDefault()` বুঝি
- ☐ `stopPropagation()` বুঝি
- ☐ Event propagation বুঝি
- ☐ Bubbling বুঝি
- ☐ Capturing বুঝি

- [] Target phase বুঝি
- [] `capture: true` বুঝি
- [] Event propagation order বুঝি
- [] `removeEventListener()` -এর function reference rule বুঝি
- [] Event delegation বুঝি
- [] `closest()` দিয়ে delegation করতে পারি
- [] Dynamic elements-এর সাথে delegation ব্যবহার করতে পারি
- [] `once` option বুঝি
- [] React event handling-এর সাথে basic connection বুঝি

Section 13 Complete

এখন পুরো DOM section:

```
DOM
|
├── 13.1 DOM Selection & Manipulation
|   ├── querySelector
|   ├── querySelectorAll
|   ├── getElementById
|   ├── getElementsByClassName
|   ├── innerHTML
|   ├── textContent
|   ├── innerText
|   ├── Attributes
|   ├── classList
|   ├── dataset
|   ├── DOM Traversal
|   └── DOM Creation
|
└── 13.2 DOM Events & Event Propagation
    ├── addEventListener
    ├── removeEventListener
    ├── Event Object
    ├── target
    ├── currentTarget
    ├── preventDefault
    ├── stopPropagation
    ├── Bubbling
    ├── Capturing
    ├── Target Phase
    └── Event Delegation
```

🧠 Final DOM Mental Model

```
HTML
↓
DOM Tree
↓
SELECT
↓
READ / MODIFY
↓
CREATE / REMOVE
↓
EVENT
↓
CAPTURE
↓
TARGET
↓
BUBBLE
↓
HANDLE USER ACTION
```

এটাই Vanilla JavaScript-এর DOM-এর core foundation ।